

Regression Test Selection in Test-Driven Development

Zohreh mafi

Sharif University of Technology

Seyyed-Hassan Mirian-Hosseiniabadi (✉ hmirian@sharif.edu)

Sharif University of Technology

Research Article

Keywords: Software Testing, Test-Driven Development (TDD), Regression Test, Program Differencing, Segmentation, Change Detection.

Posted Date: March 23rd, 2023

DOI: <https://doi.org/10.21203/rs.3.rs-2707846/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License. [Read Full License](#)

Additional Declarations: No competing interests reported.

Version of Record: A version of this preprint was published at Automated Software Engineering on December 27th, 2023. See the published version at <https://doi.org/10.1007/s10515-023-00405-w>.

Abstract

The large number of unit tests produced in the test-driven development (TDD) method and the iterative execution of these tests extend the regression test execution time in TDD.

This study aims to reduce test execution time in TDD. We propose a TDD-based approach that creates traceable code elements and connects them to relevant test cases to support regression test selection during the TDD process.

Our proposed hybrid technique combines text and syntax program differences to select related test cases using the nature of TDD. We use a change detection algorithm to detect program changes. Our experience is reported with a tool called RichTest, which implements this technique.

In order to evaluate our work, seven TDD projects have been developed. The implementation results indicate that the RichTest plugin significantly decreases the number of test case executions and also the time of regression test. The test suite effectively enables fault detection because the selected test cases are related to the modified partitions. Moreover, the test cases cover the entire modified partitions; accordingly, the selection algorithm is safe.

The concept is particularly designed for the TDD method. Although this idea is applicable in any programming language, it is already implemented as a plugin in Java Eclipse.

1 Introduction

Test-driven development [1] is one of the agile defect-reduction practices. In this strategy, software development is accomplished by using the “test-first” approach in which the test is written before the source code. The code is only written when test failures occur. In the TDD method, therefore, the writing of extra code is avoided, and the number of test cases grows rapidly. One of the key aspects of TDD is refactoring [2] which improves the software design and code structure quality.

The TDD method has drawn the interest of software developers because of its advantages, including short and simple readable code, high-quality code, reliability, maintainability, and the capability of regression testing (as a result of creating a set of unit tests). Apart from its advantages, TDD also has certain deficiencies such as higher development time [3]. This study aims to resolve one of the drawbacks that has been less considered previously—the large number of test cases and the necessity of repeated executions.

The number of test cases generated in TDD is greater than that of other methods [4]. As a result, the time required for the regression test increases significantly. On the other hand, it is necessary to re-execute all of the test cases after each modification to ensure that the code remains accurate thereafter. A substantial amount of time is subsequently required in order to execute the test cases in the TDD method.

There are many cost reduction algorithms reducing the number of test cases, which we will discuss in Sections 2.2 and 2.3. Different techniques may have different performances in different environments. The suitable technique is therefore selected based on methodology, topic, and program conditions. However, none of these methods are specifically designed for TDD. Therefore, this research proposed a test selection algorithm for TDD implemented programs to reduce the regression test execution time in TDD. Our experience is reported with a tool called RichTest, which implements this technique. It is a Java plugin and is available as a GitHub project[1].

Textual differencing is not based on programming language, but we use a hybrid technique that combines text and syntax program differences to detect code changes, so it is necessary to choose the programming language. Since Java is one of the three most popular languages in the last twenty years[2] and has been widely used, this language was considered as a reference language.

We use a hybrid differencing technique as well as using block concept to divide the program into small trackable elements. Segmentation is defined on two levels. High-level blocking considers each methods as a block, low-level blocking considers each statement, such as an if statement, as a block

After adding a new test case, we run that test case. If the test case passes, then the next test case will be added, but if the test case does not pass, the source code must be modified to pass the new test case.

- RichTest performs code segmentation to track code elements. It creates both code and test blocks.
- RichTest identifies all modified code blocks.
- RichTest connects modified code blocks to the new test case that leads to these changes.
- In the test selection phase, RichTest tracks and selects only those test cases that are related to the modified parts of the code, so instead of running all the test cases, only the selected test cases run.

Section 2 discusses the basics and the principles of TDD as well as the regression test, which must be run repeatedly in the TDD cycle. Program differencing as one of the regression test selection methods used in this article is presented in detail and a comparison between different levels of its implementation will be provided. Section 3 introduces related work.

In Section 4, our test case selection algorithm will be discussed in detail. Segmentation, segment comparison, and relationship creation algorithm are explained in this section. The RichTest tool, which is developed to implement the foregoing is explicated in Section 5. Automatic and manual block segmentation and regression test wizard are explained in this section.

Section 6 presents the evaluation of RichTest using another program that has been developed to access the TDD projects on GitHub to compare the number of executed test cases in TDD and RichTest. Section 7 concludes the paper. Several images of the RichTest tool are illustrated in the appendix A.

[1] <https://github.com/MafiZo/RichTest.git>

[2] <https://www.tiobe.com/tiobe-index/>

2 Background

The proposed technique allows for avoiding the execution of some test cases in TDD. This section discusses the basics and principles of TDD as well as its advantages and disadvantages. The regression test must be repeatedly run in the TDD cycle. Previous work on the regression test and the principal approaches for its cost reduction, particularly program differencing from the standpoint of regression test and other software maintenance applications, is presented in detail.

2.1 Test Driven Development (TDD)

In the traditional approach, software development proceeds by first creating the working code and thereafter writing unit tests [5]. This method is sometimes referred to as test-last development. In several traditional software development models, such as the waterfall model, software testing is one of the last tasks to be performed before the software maintenance phase. On the contrary, in modern and agile software development methods, testing is often adopted as an integrated part of the entire development process. This technique aids developers in finding and fixing bugs starting from the early phases of development. In test-driven development, however, software tests are written before the actual source code [1].

The concept of the TDD method was first studied by Beck [1]. As its name suggests, TDD is a software development approach for building software incrementally in which tests are written before the production code and allows the test cases to drive the production code development. The code is written only when test failures occur. As a result, the tests are always written first and, thereafter only a sufficient amount of code is written to pass them [1], [6]. Despite its name, TDD is not a test method; it is in fact a new software design and implementation method in which the idea of writing tests before developing the code is combined with the concept of refactoring [2].

According to Astels, in the TDD method, the project is first broken into smaller parts using the divide and conquer method. The program is developed incrementally, starting from the development of each part by writing a test [7]. The TDD process proceeds as follows [1], [6]:

1. Add a small test;
2. Run all tests and see if the new one fails (The test might not even compile);
3. Write a minimum amount of code to pass the test;
4. Run all tests and see all them succeed;
5. Refactor the code to clean them and remove possible duplications.

The development process is thereafter continued by repeating the steps mentioned above.

2.2 Regression Test (RT)

In the software development and maintenance process, product requirements are modified or corrected because of the addition of new customer requirements. These changes are implemented to match new technologies and environments, fix hidden errors that occur in various stages of development, and fix deficiencies and bugs to improve current features.

RT is an activity that is performed after a change is implemented in the system. Its objective is to reveal the defects that may have been introduced by these changes as a result of software evolution [8]. In view of the large number of test cases, RT is extremely time-consuming. It is therefore an expensive test to validate the modified software. To reduce cost, several techniques may be employed. The four principal cost reduction approaches are (1) RT minimization, (2) RT prioritization, (3) RT optimization, and (4) RT selection [9]. The coverage-based RT using program differencing used in this paper can be considered as an RT selection method.

2.2.1 Regression Test Minimization (RTM)

According to [10], RTM refers to the removal of redundant test cases from the test suite. Minimization is sometimes also called test suite reduction, meaning that the elimination is permanent.

2.2.2 Regression Test Prioritization (RTP)

Test case prioritization aims to reorder test cases to increase the rate of fault detection during RT. The RTP prioritizes tests based on error detection criteria or code coverage using experimental methods. Thus far, various prioritization strategies have been suggested [11].

2.2.3 Regression Test Optimization (RTO)

RT techniques are considered from the point of view of multi-objective optimization and Artificial Intelligence (AI). Their main goal is to select test cases through the use of optimization or AI approaches. Some of the RTO techniques are based on fuzzy logic, and some of them are based on heuristics. This technique includes contributions in the line of greedy algorithms, Pareto optimization, and integer linear programming in combination with genetic algorithms [9].

2.2.4 Regression Test Selection (RTS)

The RTS method chooses some of the test cases and ignores the rest. In this category, the reduction is also present but its strategy focuses on the detection of modified parts of a program that normally runs based on white box static analysis [9].

Safe RTS techniques prove that under certain well-defined conditions, test selection algorithms exclude no tests (from the original test suite) that if executed would reveal faults in the modified software. Under these conditions, the algorithms are safe, and the fault detection abilities are equivalent to those of the retest of all tests. [12].

2.3 Program Differencing

In regression tests, the knowledge of which parts of the program are unmodified can aid in identifying the test cases that do not have to be executed [13]. Considering the fact that the behaviors of preserved components in the new and old versions of a program do not differ at runtime, it is guaranteed that no retest of all cases is necessary, and testing the affected component only is sufficient [14].

Program differencing is also a principal step to solve some of the crucial problems in software maintenance such as locating bugs, introducing changes, tracking code pieces or drawbacks in versions, merging files, and analyzing software evolution [15]. In general, modifications in software products are classified into three levels: textual modifications, syntax modifications, and semantic or behavioral modifications. The previous works are presented in these three categories.

2.3.1 Textual Differencing

In the textual approach, regardless of whether the code file is an executable program, the common parts of the two versions are identified using algorithms, e.g., “longest common sub-series algorithm.” For instance, diff [16] is among the most utilized tools in UNIX that presents the difference between two versions of a program. It generates a report consisting of a series of added or deleted lines between two files after identifying the common parts.

Vokolos et al. [17] developed a tool for textual differencing, named Pythia, which is capable of analyzing large software systems written in C. The results indicate that this technique is considerably fast and can significantly reduce the size of RT suite.

An enhanced language-independent tool, LDiff [18], is developed based on Unix diff and resolves numerous problems encountered by the latter. These include determining if a line has been modified or is a result of additions and deletions, and tracking code blocks that have been moved up or down inside the file.

Another tool that tracks source code lines between two different versions of the file is LHDiff [15], which takes two different versions of the program as input and uses the Unix diff technique to identify unmodified parts. In order to track the remaining lines, a mixture of context and content similarities is used.

2.3.2 Syntactic Differencing

Yang [19] obtained the difference between the two programs based on grammar and parse trees. This is known as the syntactic difference. Each program is displayed using a parse tree built by the parser. The tree-matching algorithm takes two trees as input and finds a set of pairs of nodes in which each node belongs to one tree and appears maximum in one pair.

Maletic [20] presented a syntactic differencing approach to analyze source code differences. The meta-differencing approach attempts to automatically produce some information related to the difference between the two programs. Complex questions on the difference between two versions of a program can be solved by this system. Meta-differencing uses an XML-based language called SrcML to display the two programs and their differences.

Archambault [21] took the graphs of two versions of a program and merged them based on similar node names to obtain a new graph. In order to reduce the graph size, the concept of MetaNode for collecting the nodes is employed. The betweenness centrality measure is used to determine the difference between the two input graphs. This value is determined for all graph nodes. The small and large values indicate the stability and instability, respectively, as well as the difference among the points.

Goto [22] considered merging similar programs to increase program maintainability and focus on structural differences. The AST trees for two similar methods are first built using Eclipse JDT; the differences among the trees are then determined. Finally, coherent code pieces are identified as Extract Method (EM) candidates. The FTMPATool is implemented to accomplish this task.

The ChangeScribe [23] tool is an Eclipse plugin that considers the textual differences between the new and previous version of the program at commit time and generates messages to automatically explain the modifications. ChangeScribe is currently applicable for Java projects on GitHub. Shen et al. [24] continued this work by defining four types of changes to describe the code change and include information that explains the reason for the code change.

The LSDiff[3] [25] tool attempts to answer some of the high-level questions of programmers and present systematic structural differences as logical rules. LSDiff represents each version of the program using a set of predicates that describe code components, their relationships, and their structural dependencies.

Falleri et al. [26] employed the GumTree tool, which is comprised of two sequential steps, to compute the mappings between two ASTs: (1) top-down greedy algorithm for finding isomorphic subtrees, and (2) bottom-up algorithm to detect corresponding nodes.

The SEGMENT tool [27] divides the different parts of the program by adding blank lines to increase the readability of the program. SEGMENT uses the program structure AST tree as well as the name information and identifies meaningful primary blocks with a particular logical operation. In order to identify logical blocks, three main types of blocks are considered: syntactically the same, data flow chain, and extended SWIFT[4].

2.3.3 Semantic Differencing

Horwitz [28] used a program graph representation and a partition operator on these graphs to semantically find differences. His partitioning algorithm is limited to a language with scalar variables, conditional statements, assignment statements, while loops, and output statements.

Binkley [14] reduced the RT cost by using semantic differences between the two programs. In his work, the limitations of program statements are reduced compared to those in [28]. He also included function definitions and function calls. He used a system dependency graph instead of a flow control graph that avoids unnecessary dependencies among the components on a path in a control flow graph. Binkley reduced the complexity of test cases using the program slicing technique.

Neamtiu [29] proposed a tool to rapidly compare the source code of different versions of C programs and thereafter find semantic differences among program versions based on partial AST matching. The tool can track simple code-level modifications related to changes in global variable names, types, and functions. This tool compares the body of functions with similar names considering that the name of function is not changed throughout the software lifetime.

Apiwattanapong et al. [13] presented a method to compare object-oriented programs and used an extended control flow graph (ECFG). Görg [30] extended the method proposed in [13] in such a way that it also supports the new concepts introduced by aspect-oriented programs.

The patent in [31] presents a technique for identifying the differences between two graphic programs. BinHunt [32] is aimed at identifying the semantic differences in the binary code between the two programs that can be used in cases where the program code is not available. BinHunt uses the STP[5] theorem proving and symbolic execution to compare the primary blocks. It is applicable only for minor differences.

Wang et al. [33] used normalized[6] control dependence trees to represent two versions of the program and improved the traditional metrics-based and graph-based approaches to propose a combinational approach.

Liu et al. [34] produced a plagiarism detection tool called Gplag. Plagiarized codes are often modified for deception, and identifying such codes is possible by using a suitable and similar code identification tool. This approach represents the program code as program dependence graphs (PDG) and identifies similar code based on the sub-graph isomorphism test.

Nguyen [35] proposed the iDiff tool as a plugin in Eclipse for identifying program differences. The iDiff can identify changes in classes and methods, track re-ordered, relocated, and renamed classes and methods, and detect internal changes in methods. The iDiff uses JavaModel and ASTParse related to the JDT plugin in order to parse the project for obtaining all information related to the types and limitations of methods.

2.3.4 Summary of Program Differencing

Table 1 summarizes the above references related to program differencing according to the type of difference identification (text/ syntax/ semantic) and tool produced. Some of these tools are related to a particular language, developed for multiple languages, and not language-dependent. Some of them normalize the code before identifying the differences and use a limited set of statements for simplification. Most of the tools use graph or tree structures.

Table 1: Comparison of Program Differencing References

Reference	Type	Language	Limit	Normalize	Representation	Description
diff [16]	Text	Independent	X	X	Text	add and delete only
Pythia [17]	Text	C	X	X	Text	add and delete only
Ldiff [18]	Text	Independent	X	X	1:1 Line	add, delete, update, and relocate
LHDiff [15]	Text	Independent	X	✓	Neighbor Lines	add, delete, update, and relocate
[19]	Syntax	C	X	X	AST	AST and synchronous pretty-printing
[20]	Text & Syntax	C/C++	X	X	SrcML[7]	High Level
[21]	Graph	Independent	X	X	Graph	Meta node
FTMPATool [22]	Syntax	Java	X	X	AST	
ChangeScribe [23]	Text & Syntax	Java	JGit	X	Code Change NLP Impact Analysis	Add comments add, delete, and update
[24]	Syntax	Java	X	X	JGit System Version Control	Add why and what comments
LSDiff [25]	Syntax	Java	X	✓	predicate	
GumTree [26]	Syntax	Java	X	X	AST	Relocate
Segment [27]	Syntax	Java	X	X	AST	basic block detection
[28]	Semantic	C	✓	✓	PRG[8]	safe set
[14]	Semantic	independent	✓	X	PDG[9]	More complete than [28]
a tool [29]	Semantic	C	X	X	AST	Semantic-preserving transformations
JDiff [13]	Semantic	Java	X	X	ECFG[10]	Difference testing
AJDiffer [30]	Semantic	AspectJ	X	X	ECFG	
[31]	Semantic	graphical	GUI	X	matching matrix	graphical programs
Binhunt [32]	Semantic	Binary Code	X	X	CFG[11] and CG[12]	Small difference , Low speed
[33]	Semantic	C	X	✓	PDG and AST	Augmented graphs
GPlag [34]	Semantic	C/C++, Java	X	X	PDG	Plagiarism detection
iDiff [35]	Semantic	Java	X	X	JavaModel	JavaModel, ASTParser attributed graph
RichTest (Our Work)	Text & Syntax	Java	SWIFT	X	AST, JSON	add, delete, update, and relocate, Reduce RT

Graph-based methodologies consider both syntax structure and data stream as abstraction levels, making those suitable bases for identifying similar code on a semantic level. Sometimes, however, problems, such as code diversity, hinder the identification of similar codes. High computational complexity in graphs limits graph size. Some studies have attempted to resolve this problem by forming meta-nodes and reducing the number of graph nodes [21]. A tree, as a special form of graph, reduces computational complexity. In particular, the use of AST trees neglects certain basic differences by considering the syntax structure [19], [22], [26], [27], [29], [33], [35]. We also use the AST tree as the base of our change detection algorithm.

Each article examined for this research has certain deficiencies. For example, some do not thoroughly discuss language statements [28], exhibit certain limitations [23], or encounter computational problems as the program grows larger and the number of graph nodes increases [32]. Some do not capable of tracking the relocated code or matching a single line of code with multiple lines with the same meaning [18]. Others do not detect the updated code and only detect lines that are either added or deleted [16], [17]. There are those that require a pre-processing phase to normalize code [15], [28], [33]. Additionally, most of the programs have high time complexities in the order of $O(n^3)$ or $O(n^2)$. The idea presented in this paper overcomes some of these limitations and its time complexity is $O(n)$. Table 2 compares the three types of program differencing (text/ syntax/ semantic).

Table 2: Comparison of the Three Types of Program Differencing

Factor \ Type	Textual	Syntactic	Semantic
Speed	Fast	Not Fast	Not Fast
Accuracy	High	Medium	Medium
Readability	Low	High	High
Scalability	High	Low	Medium
Flexibility	High	Language Dependent	Language Dependent
Abstraction Level	Line-Based	Statement-Based	Module-Based (Function-Based/Class-Based)
Regardless of worthless details	Low	High	High (ignore refinements)
Modification Level	Add, delete, (update)	Add, delete, update, relocate	Transformation
Representation	Line, Text	AST Tree	Graph, Tree

Textual differencing can be applied to any text file. It indicates detailed changes such as added or deleted or updated lines. Its line-based view does not respect syntactic boundaries. Thus, the differences often do not sufficiently reflect on the real meaning of the changes and often are not readable enough, also relocating the code may be unsupported.

Syntactic differencing is based on grammar and parse trees, therefore it ignores changes to whitespace, comments, and preprocessor statements. Tree-matching algorithms are used to identify unchanged parts of the tree (code) and display the remaining parts as syntactic differences. These algorithms are generally slow and thus do not scale to large systems. Also, sometimes two completely identical structures may be in different situations that show different functionalities and are not semantically the same.

Semantic differencing corresponds to changes in the program functionality and is not related to programming structure or statements. Normalization methods are usually used in order to remove code variations. Module signature modification is considered as a semantic difference.

- [3] Logical Structural Diff
- [4] statements such as synchronized, do, try, for, if, while, and switch
- [5] Simple Theorem Prover
- [6] Code normalization is a semantic-preserving transformation
- [7] Source Code Markup Language
- [8] Program Representation Graph
- [9] Program Dependence Graph
- [10] Extended Control Flow Graph
- [11] Control flow Graph
- [12] Control Graph

3 Related Work

First, previous works on the TDD are examined and different approaches are considered. The various methods that have been suggested are studied to reduce the RT execution time and to propose a suitable method to reduce this time in the TDD method.

As an instance, Continuous Test-Driven Development (CTDD) recommends background testing to reduce this time. CTDD is a recent enhancement of the TDD practice and combines the TDD with continuous testing practice. During the execution of test cases, the developers have to stop the system to execute the test physically, thus increasing the program development time. By using the continuous compile feature in the new IDEs, e.g., Eclipse or Visual Studio that keep the source code in the compiled mode, this goal of reducing execution time will be realized [36].

Madeyski et al. [37] evaluated the CTDD practice via an empirical study in a real industrial software development project that employs Microsoft.NET. If the developers that use TDD adopt CTDD, it can run slightly faster, thereby leading to slight improvements in coding. Although the idea is to write a code and execute the test in parallel, it does not change the number of test cases and the number of times they run; hence, it does not reduce the amount of load and processing costs. In terms of reducing the number of test cases, our proposed method is thus preferred.

In another instance, Cibulski et al. performed the RT in TDD. A small subset of test suites for each small local change is automatically found. The objective of this study is to identify 20% of the tests that can detect 80% of errors instead of creating an infinite subset of tests that detect 100% of errors [38].

Different from other safe selective RT methods, this technique limits the number of selected test cases. Results show that the test suite is not safe. Results show that the test suite is not safe because 20% of the errors were ignored. The restricting method reduces this problem to a prioritization problem, which chooses 20% of the higher-priority test cases.

Cibulski presented selection techniques based on natural language analysis and dynamic programming via the TestRank tool. TestRank takes a Java program with its test suite as input and requires a pre-processing step, which is considerably time-consuming. As mentioned above, two fundamental problems arise: (1) the test suite is unsafe, and (2) the synchronization of the system with the latest version of the program is considerably time-consuming (up to one day, 24 hours).

As another related work, we refer to ChangeScribe [23] and iDiff [35] tools, which are Eclipse plugins similar to our project. These plugins generate comments to explain changes. ChangeScribe only considers the textual differences of the new program from the previous version and generates comments that explain changes. ChangeScribe, however, cannot be used for RTs and is only applicable for Java projects existing on GitHub because it does not have a version manager. The iDiff tool receives two program versions at a time and determines the modified, deleted, or added classes and methods. It does not provide, however, a complete environment that contains all versions created throughout the software evolution process. Also, Eclipse has been considered in [39] for learning techniques selection.

4 Our Test Case Selection Algorithm

In the TDD method, any minor changes result in RT. The problem, therefore, is the growing number of tests and the necessity of re-executing these tests. Finding a small subset of the test suite that can be utilized to scrutinize the software with high confidence is thus important.

4.1 Add a New Phase to Three Phase TDD Cycle to Reduce the Test Re-Execution Time

As pointed out in [40], reducing the time of test execution differs among various software development methodologies, so a TDD-specific approach should be determined to choose test cases that must be re-executed in each iteration of the TDD process.

In pure TDD, the part of the code that each unit test belongs to is precisely determined. The code is developed after writing the test; hence, there is a close relationship between the unit test and the modified code. In every step of the software development process, the modified parts of the code are determined, and only tests that lead to these parts are chosen for re-execution.

Fig. 1 illustrates the TDD tasks that are comprised of three steps, which correspond to the three phases of the TDD cycle. In the first step, the new test is written and executed until an error occurs. In the second, the code is written to pass the test. In the third, the refactoring phase occurs.

Fig. 2 illustrates the tasks in our improved TDD cycle that are comprised of four steps. The first and second steps are similar to the first two steps illustrated in Fig. 1. In these two steps, however, only “the new test” is executed instead of executing “all test cases”. The third step is a new phase added to this figure. In this step, tests that require re-execution are selected and executed using our selection algorithm. The last step in both figures is refactoring. In the refactoring phase of Fig. 2, only tests that are related to the modifications are selected and executed. In the improved TDD, test case execution is limited in all of the given steps, as illustrated in the flowchart in Fig. 2.

4.2 Segmentation

First of all, we divide the program into several code blocks based on the Java programming language grammar. Program segmentation has three benefits:

1. The program is divided into small independent components called blocks.
2. Each block has a fixed unique name, so it can be traced. Line tracking is not applicable. Because the program changes and as a result the line number also changes.
3. It is possible to detect changes in the program by detecting changes inside the block. Also, the location of changes in the program is specified precisely.

We desire two levels of granularity for these code blocks: (a) coarse-grained level for whole classes and methods and (b) fine-grained level for language control flow statements. However, structured block information is stored in a database.

By segmenting the program code into blocks and assigning a name to each block, code tractability property is created, so any movement or update in the block content will therefore modify the program code in that block. This determines the location of changes and makes block relocation traceable.

4.3 Change Detection Algorithm

We initially decided to compare the block content textually. Textual-differencing approaches are limited to a line-level granularity. We omitted extra spaces between words and lines, as well as entire comments, then we compare this pre-processed text of each block with its previous version to detect if it has changed. Later, however, we also decide to use an abstract syntax tree to compare the contents of each block. By applying this structure, minor changes can be ignored too. So, we use the combination of text and syntax differencing method. The difference between the two versions of a program is determined by identifying the modified code blocks based on Java grammar as a combination of textual and syntactical difference methods.

Although semantic and behavioral modifications are at a higher level and indicate real changes, the focus of this study is on textual and syntactical modifications. The reason behind this choice is that we have to find all the tests that require re-execution after code modifications. In the case of omitting tests that check the changes in appearance (e.g., change in the name of a variable or method), the set of test cases is not considered safe. Hence, although the modifications are of the refactoring type, the tests should be re-executed to ensure accuracy. Focusing on the textual and syntactical levels may ensure the safety and reliability of the RT.

4.4 Relationship between Test Case and Code Blocks

After adding any new test case that has encountered errors, new code blocks are created, or existing code blocks are modified. These modifications are implemented to pass the last test; therefore, the last test is related to the modified code block(s). A connection must therefore be automatically established between the modified code blocks and the last test case to be used by the selection algorithm.

Given project P , includes a set of code blocks C and a set of test cases T . To pass the new test case t , some of the code blocks $M \subset C$ will be modified (to $M' \subset C'$) and new code blocks N may be created. So the new version of project P' consists of C' and T' such that:

$$C' = C - M \cup (M' \cup N) \tag{1}$$

$$T' = T \cup \{t\} \tag{2}$$

We define **Link** relation as follow:

$$\textbf{Link}: T' \times C' \tag{3}$$

$$\forall c \in (M' \cup N), \textbf{Link}(c, t) \tag{4}$$

4.5 Test Case Selection

In the TDD method, the code is written or modified only because of test failure. In our proposed concept, however, the failed test is connected with modified code blocks. This task is iteratively executed, and the connections between the code blocks and related unit tests are established and tracked. In order to run the RT, the test cases connected to modified or newly added code blocks are chosen as candidate unit tests for execution.

As a result, the iterative execution of test cases, which are not connected to the modified parts of the code, is avoided, and the number of selected test cases is reduced.

After specifying the ‘Start’ and ‘End’ versions of the program for RT, the latest commit[13]s and new test cases are identified in this interval. All code blocks related to the new tests are specified, and the tests relevant to these code blocks are introduced as candidate tests. Fig. 3 illustrates our improved test selection algorithm.

As shown in Fig. 3 the RichTest built-in version manager lets the custom start and end version, not necessarily consecutive version, although it is set to the last two versions by default.

4.6 Test Case Selection Example

Our test case selection algorithm is presented using the following example.

Example 1:

Suppose that test cases Ta001–Ta010 are written in sequence. In order to pass each test, code blocks Ca001–Ca007 are added or modified, as listed in Table 3. Ta005 and Ta008 pass immediately without changing the code, but the rest of the test cases cause changes in some code blocks and a new commit is generated. Commits c1-c8 shows all the saved program copies.

A question then arises: from the commit related to Ta008, i.e., c6–c8, which test cases are selected for the RT?

Table 3: Relationship of Test and Code in Example 1

Commit	Test Block	Code Block(s)
c1	Ta001	Ca001
c2	Ta002	Ca001
c3	Ta003	Ca001, Ca002
c4	Ta004	Ca003
c4	Ta005	—
c5	Ta006	Ca004
c6	Ta007	Ca003, Ca005
c6	Ta008	—
c7	Ta009	Ca001, Ca006
c8	Ta010	Ca001, Ca007

It can be observed that Ta008–Ta010 are new tests in this interval that are related to Ca001, Ca006, and Ca007 code blocks, respectively. These code blocks are connected to Ta001, Ta002, Ta003, Ta009, and Ta010 test blocks (as shown in Table 4) that are candidates in the RT.

Table 4: Modified Code Blocks in Example 1

Code Block	Test Block(s)
Ca001	[Ta001 , Ta002, Ta003, Ta009, Ta010]
Ca006	Ta009
Ca007	Ta010

Although T8 is recently added, its re-execution is unnecessary because this test previously passed without making any code modifications. To ensure safety, however, this test is still considered.

[13] Each copy of the program a developer saves. It is not necessarily a new issue/version of the program.

5 Richest Plugin Overview

RichTest plugin (Rich Software Testing) is based on the Eclipse integrated development environment and is written in Eclipse version 4.8, which is recommended for running RichTest. Fig. 4 shows a snapshot of using this plugin. More additional images can be seen in the appendix A.

By installing[14] RichTest on Eclipse, the **developer** will be able to develop TDD projects faster and easier as fewer test cases are selected and executed in the development phase. It also offers several widgets[15], such as Block Information View, Commit View, Version Manager View, Regression Test View, and Compare View to facilitate the use of RichTest which is explained below.

BlockInfoView. It is possible to display the Block List and the relationship between code blocks and test blocks, as well as manage the relationship manually.

CommitView. It is possible to show all block creations and modifications and also filter all versions and commits of each block.

VerssionManagerView. It is possible to set a new version for the projects.

RegressinTestView. It is possible to automatically select candidate test cases, run them to show the time and results (Fail/Pass), and export them to an Excel file format.

CompareView. It is possible to compare two different commits of each block. The code block will be shown in two situations (before/ after) and the differences will be colored and presented on **CompareResultsView**.

Preferences[16] such as Automatic/Manual Block Selection, Code Granularity (Coarse/Fine), and Enable/Disable TDD Mode. Related figures are attached.

5.1 RichTest Plugin Working Process

RichTest segments the source code and test code into code blocks and test blocks, respectively, during the project development process. It also identifies modified code blocks in each commit, detects the relationship between test blocks and code blocks, and stores them in a database. The main purpose of RichTest is to find candidate test cases for the RT process that are made possible by the connections already made between test blocks and code blocks.

5.1.1 Automatic Block Segmentation

The segmentation process can be implemented both manually and automatically. In the automatic mode, whenever a file is stored, the plugin segments the file contents into blocks, adds new blocks, and updates modified ones. There are two types of blocks: test block and code block.

1. Test block is in fact a complete test case. It is considered as a block only because of its similarity to the code block.
2. The code block is determined based on the structure of the programming language instructions. Each block represents a node in AST[17].

Automatic test block segmentation detects the “@Test” annotation to identify each test block, and automatic code block segmentation is based on the AST. The code block granularity degree can be chosen from two levels: (a) coarse-grained level for classes and methods and (b) fine-grained level for language control flow statements (SWIFT instructions[18]). The first level produces larger and fewer blocks, and the second level produces smaller and more blocks, especially in large projects. The automatic code block segmentation activity diagram is shown in Fig. 5.

During segmentation, a unique name is automatically assigned to each new block. The block nomination method varies depending on whether the block is a code block or a test block. The names of code and test blocks follow the LNC and LNT regular expressions, respectively.

$LNC = 'C' lddd \quad (5)$

$LNT = 'T' lddd \quad (6)$

$l ::= a|b|c|...|z|A|B|C|...|Z \quad (7)$

$d ::= 0|1|2|...|9 \quad (8)$

5.1.2 Manual Block Segmentation

Segmentation can be manually performed by the developer. Using RichTest, any valid arbitrary part of the code could be specified as a block by simultaneously selecting the desired part of the code and pressing CTRL+1 Keys. The manual code segmentation activity diagram is shown in Fig. 6

5.1.3 Difference Detection Algorithm

The RichTest tool transforms each code block into a JSON array. In order to identify the differences in each code block, the elements of the JSON array are compared with those of the previous state. If there is a difference among the array elements, then this block is recognized as a modified block, and the block contents and properties are updated in the database. The JSON is a structured textual format for holding the information that ignores ineffective textual modifications (e.g., adding comments).

The primitive version of the plugin has no programming language limitation and is capable of supporting all languages supported by Eclipse because it uses a text-based difference algorithm. The new version of the plugin, however, is only applicable to the Java programming language because it detects differences using the AST based on Java grammar and stores the syntax information of blocks.

In the new version, the comparison method is a combination of both textual and syntactic differencing methods. Segmentation is performed based on Java syntax, and the block content is stored in the AST model. The data values are compared based on their textual contents.

As emphasized in the literature review, the use of each of the existing methods to find textual and structural differences has advantages and limitations. In this study, these two methods are combined to exploit the following advantages: precision and speed in textual difference, code relocation, and ignoring insignificant modifications in a syntactical structure. The textual difference related to each small modification is considered in the AST to ensure that no related test is ignored in the test case selection process.

5.1.4 Connecting Code Blocks to Test Blocks

Each code block can be connected to one (or more) test block(s). In the manual mode, the block relationships can be manually managed using the “Block Information View.” In automatic mode, the last test block added is automatically connected to all modified code blocks. In this mode, however, it remains possible to manually manage block connections.

Fig. 7 shows an example of the relationship between test and code blocks. A code block may be associated with none, one, or several test blocks. As shown in Fig. 7 the Ta001 test block is first written, then the Ca001 code block is generated as a result of the Ta001 test failure. Next, the Ta002 test block is written; subsequently, there is a change in block code Ca001. To pass the Ta003 test, block code Ca001 is modified again. The Ta004 test block generates the Ca002 code block. The Ca003 code block is created after the Ta005 test block failure.

Inside an existing function, a new loop statement may be added that can be defined as a new code block. In this case, the internal block is a part of the external block, and the test connection to the internal block also extends to the external block. The Ta006 test block is, directly and indirectly, related to Ca004 code block and external Ca003 code block, respectively.

After each newly added test fails, new code block(s) are created, or existing code block(s) are modified. These changes are necessary to pass the last test. Semantically, the given test is relevant to these modified code block(s). A link is therefore created from each of the modified code blocks to the last test; this connection is stored in the database. Fig. 8 shows how the connection between code blocks and test blocks is established.

5.2 Regression Test Wizard

“Regression Test Wizard” produces a list of candidate test cases between the “Start Version” and “End Version” of the program. The wizard also assigns a name for the list. The last and previous versions are considered as default for the End and Start versions.

After specifying the desired Start and End versions, recently added test cases are highlighted, and all test cases associated with the modified code blocks are also nominated. Only candidate test cases are shown. These can be saved and run, as shown in Fig. 9.

After the execution of test cases, successfully passed and failed test cases are determined. The passed tests are identified in green with a “success” result tag, whereas the failed tests are identified in red with a “fail” result tag. The runtime information of each test case is in milliseconds. Candidate test case information can also be viewed and executed through “Regression Test View.”

[14] Help-> Install New Software, and also should set Window-> Preferences as Dependency folder address

[15] Available from Window->Show View ->Other -> RichTest

[16] Available from Window-> Preferences->RichTest

[16] Abstract Syntax tree

[17] switch, while, if, for, foreach, and try

6 Empirical Evaluation

For the preliminary evaluation, RichTest is employed in three simple examples: exponentiation (power), array selection sorting, and linked list that calculates an integer number raised to the power of a positive integer, sorts array elements in ascending order, and creates and modifies linked lists, respectively. These three programs were written step by step according to TDD kata [41] when the RichTest plugin had not yet been implemented by one of the authors. Exactly the same process was re-implemented with RichTest after implementation by another authors.

“Re-implementation” is the same process as implementation, except that it is done in the presence of the RichTest plugin to automatically perform some tasks such as code segmentation, difference detection, relation creation, and test case selection.

Since our goal is to measure the effectiveness of the tool, we kept all the conditions constantly except the implementation environment. For this purpose, we added the same previous test cases one by one and wrote the same previous codes utilizing RichTest. This plugin reduces the number of execution of test cases by selecting some of the test cases. Four large projects are also implemented with and without RichTest tool. Full details are presented in subsequent sections.

6.1 Small Program Development using RichTest

The three small programs—Power, selectionSort, and linkedList—are implemented in the Java programming language using the TDD method twice, with and without utilizing RichTest. Power, selectionSort, and linkedList programs were implemented by five, ten, and nineteen test cases respectively. The two first implementations took five steps, so they have five versions. The last one was implemented in ten steps, so it has ten versions. The implementation results are summarized in Table 5.

Table 5: Comparison of number of test executions in TDD and RichTest (three simple programs)

Factor \ Program	Power		selectionSort		linkedList	
	TDD	RichTest	TDD	RichTest	TDD	RichTest
Total number of executions (sum)	61	6	68	9	387	25
Average= sum/v	12.2	1.25	13.6	1.8	38.7	2.5
Number of tests (n)	5		10		19	
Number of versions(v)	5		5		10	

It is predictable that the total number of tests performed in the TDD method is more than our method. Because we select some of the test cases, while traditional TDD, executes all of them. But the difference between these two methods is huge. It is trivial that as the program grows larger, the number of commits also increases; consequently, the advantages of RichTest become more evident. The RichTest plugin successfully reduces the selected test cases by reducing the number of test cases and the number of times each test is executed.

6.2 Large Project Development using RichTest

In order to evaluate RichTest with large and real programs and identify projects based on the TDD in GitHub, a survey is conducted using a new program. Similar to the work of Borle [42], this program searches GitHub for projects that contain created test files before project development or at least one week thereafter.

6.2.1 TDD Projects on GitHub

To compare the plain TDD method with the suggested improved technique, some real TDD Java projects are selected from GitHub. Although GitHub provides a code repository for projects, it is not possible to determine the development process of projects. On the other hand, there is no precise definition for TDD projects. It is also not possible to determine with certainty whether the project follows the TDD process using a project repository. Borle [42] formulated a method for detecting TDD projects on GitHub; however, the names of discovered projects were not disclosed. The authors acknowledge the uncertainty of results with respect to the foregoing problems and attempt to construct a range of code repositories that shows the extent that the TDD process is employed in their projects.

We implemented a Java script program that includes ten asynchronous and normal functions to crawl GitHub repository. First, it creates an asynchronous iterator over all public repositories of GitHub that have Java listed as one of their languages. Then it filters the returned values, limiting them to repositories that have all the following specifications:

1. Primary Programming Language: Java
2. Size > minSize
3. No. of Commit > minNoCommit
4. No. of TestFile > 0
5. (TestCreateDate < CodeCreateDate) or ((TestCreateDate < 30th CommitDate) and (TestCreateDate < CodeCreateDate + 1 week))

This program is employed to find the TDD projects on GitHub. Within one hour, 89 projects with the above-mentioned properties are identified. Four of these projects, which have a suitable number of lines and commits that could be executed in Eclipse, are chosen for evaluating the RichTest tool. These projects are ScribeJava, Jasmin-Maven Plugin, Metric-Core, Jedis, Commons-Math, and Junit-dataprovider. It should be mentioned that the programs selected as TDD projects are not necessarily TDD. These open-source projects, however, have basic TDD specifications with test files besides the code files. Their evolution process can be accessed, and hence, they can be re-implemented as TDD projects.

Table 6 summarizes the properties of these projects.

Scribejava is a simple OAuth library for Java. Jasmine-maven plugin is a Maven plugin for the JavaScript testing framework, Jasmine. The Metric-core is the central library for Metrics that provides basic functionality. Jedis is a client library in Java for Redis. It is driven by a Keystore-based data structure for persistent data and can be used as a database, cache, message broker, etc. Commons Math is a library of lightweight, self-contained mathematics and statistics components addressing the most common problems not available in the Java programming language or Commons Lang.. Junit-dataprovider is a TestNG like dataprovider runner for JUnit with many additional features.

It should be mentioned that the programs selected as TDD projects are not necessarily TDD. These open-source projects, however, have basic TDD specifications with test files besides the code files. Their evolution process can be accessed, and hence, they can be re-implemented as TDD projects.

Table 6: Properties of Selected GitHub Projects

Project Name	LOC	Number of versions(v)	Number of Tests (n)
ScribeJava [14]	10668	1134	135
Jasmin-Maven Plugin [15]	3815	635	103
Metric-Core [16]	6734	240	54
Jedis [17]	30786	1586	592
Commons-math [18]	260760	7010	3073
Junit-dataprovider [19]	22544	586	762

6.2.2 Re-Implementing GitHub Projects

After finding the appropriate repository, we re-implement each project, step by step. For each repository, we first create an empty project and transfer the first commit of the repository to this project. Then we select the "Save" button. The RichTest performs segmentation and adds start and end comments and inserts block information in the related database. This is the first version of the project.

In the next steps, we have to complete the project incrementally according to the main branch and apply the changes in each commit. We apply test files changes and then we apply code files changes. Then we select the "Save" button again. From the second version onwards, not only automatic block segmentation but also block relationship creation is done and the related information is recorded in database. It is important that in each commit the changes related to the tests are applied first and then the changes related to the code are applied so that the connection between the test cases and the modified code blocks is correctly recognized and recorded. At last, we run the RichTest Regression Test Wizard to select related test cases. Then we store the number of RichTest selected test cases as well as the total number of test cases in two separate table to calculate the total number of the executed test case in each method.

We perform this process for all versions of all projects. The number of versions in each project is extremely high. As a result, it is relatively time-consuming to repeat the process for all versions. Only 100 versions are therefore considered in the first project, and overall, fewer versions are considered in other projects (29, 28, and 15 versions were re-implemented for projects Jasmin-Maven Plugin, Metric-Core, and Jedis, respectively).

Selected projects are not originally written with our plugin; hence, the first version of some projects that have more than one test case, was re-implemented manually to establish the connection between code blocks and test blocks. Block segmentation, however, is generally automatically implemented.

Table 7 summarizes the number of versions considered in each project and the number of lines of code (LOC) in the first and last considered versions, as well as the number of test cases in the first and last desired version.

After each modification, the new version is stored, and the Regression Test Wizard is executed. Candidate test cases that are relevant to the modified code blocks are provided by the plugin. The number of candidate test cases is thereafter considered to calculate the number of times the test cases are executed.

Table 7: Number of considered versions for four open-source projects on GitHub

Project	ScribeJava	Jasmin-Maven Plugin	Metric-Core	Jedis
First LOC	1464	111	3027	220
Last LOC	2266	545	4067	830
Number of desired versions (v)	100	29	28	15
t_1^+	51	4	4	10
t_2^+	81	13	4	93

t_1^+ : Number of test cases in first version

t_2^+ : Number of test cases in our last desired version

Table 8 lists and compares the number of candidate test cases executed in TDD and RichTest plugin for these four selected open-source projects on GitHub. The result indicates that the use of RichTest plugin significantly reduces the number of test case executions by minimizing the number of selected test cases

at runtime.

Table 8: Comparison of Number of Executed Test Cases in TDD and RichTest for four projects on GitHub

Factor	Project		Jasmin-Maven Plugin		Metric-Core		Jedis	
	TDD	RichTest	TDD	RichTest	TDD	RichTest	TDD	RichTest
Total number of executions (sum)	6056	173	296	54	112	20	730	116
Average= sum/v	60.56	1.73	10.21	1.86	4	0.71	48.97	7.73
Number of test cases (n)	(51-81) [†]		(4-13)		(4-4)		(10-93)	
Number of versions (v)	100		29		28		15	

†: format (m-n) indicates that the start version of project has m test cases and the end version has n ones.

As can be seen in Table 8, the two columns TDD and RichTest have significant differences in all projects. This difference is greater for the first project (ScribeJava). We re-implemented the first project up to the 100th version. As to the other projects, a smaller number of versions were re-implemented. So, the difference between the number of times of test executions of TDD and RichTest in ScribeJava is considerably larger compared to the other projects. This difference is due to the fact that the number of versions in this project is much higher than the others and RichTest ability is more evident in the high number of versions.

However, one may argue that some TDD developers' only re-run test cases related to the new class or the new unit. So we decided to resume our work and compare our approach with such a simpler TDD. We did these reviews for four TDD projects on GitHub. The results were recorded in separate tables. The summation of run test cases was calculated. The final results are shown in Table 9. Although the simple TDD works much better than TDD, our method still performs better than the simple TDD.

Table 9: Comparison of the Number of Run Test Cases in TDD, simple TDD, and RichTest for Four Projects on GitHub

Project	TDD	STDD	RichTest	Number of test cases	Number of versions
JUnit-dataprovider	12312	4437	750	(1 - 240)	69
Commons-Math	10234	2061	501	(9-226)	70
ScribeJava	4485	257	157	(51-66)	69
Jedis	5432	1106	382	(10-136)	53

6.3 Evaluation Results

Considering that there is no standard dataset or projects to compare our method with others' methods, for the baseline we desired two methods, pure TDD and STDD. Some TDD developers only re-run test cases related to the new class or the new unit. So we decided to compare our approach with this simpler TDD we called STDD.

To evaluate the improved method, three small programs and four large open-source projects on GitHub are implemented in RichTest. The number of test case executions in the main TDD method and improved method are thereafter calculated and compared. Table 5 illustrates the number of test case executions in the traditional TDD method and improved method for the three simple programs, and Table 8 illustrates the results for four real projects.

As illustrated in Fig. 10 (obtained from Table 5 and Table 8), the RichTest plugin (box crosshatched with **orange** and diagonal lines) significantly reduces the number of test case executions by reducing the number of selected test cases at runtime. This reduction would be more significant in large projects with a larger number of test cases (ScribeJava is an evident example).

In Fig. 11, the total number of test cases is divided by the number of versions to determine the average number of test cases per iteration. As shown in this figure, in RichTest, the average number is small in all cases but varies according to the number of test cases in the TDD. This figure confirms that the average number of candidate test cases in the improved method is small and is not related to the number of test cases.

The desired versions of ScribeJava are larger compared to the other projects. As illustrated in Fig. 12, the difference in the number of test execution times between the two methods (TDD and RichTest) in this project is more significant. This indicates that the advantages of this approach are more evident in large projects that have a longer production process and when the number of test cases is higher.

Fig. 12 illustrates that the number of times that the test runs in RichTest (**orange** dashed line) completely overlaps with the number of test cases (**black** dotted line labeled as "*n*"). The number of times the test runs in the TDD (**blue** line), however, significantly differs from the number of test cases.

Fig. 13 compares the total number of run test cases in three methods, TDD (**blue** box), simple TDD (green box), and RichTest (**orange** dashed box). As shown, RichTest conquers simple TDD as well as TDD.

Due to the reduction in the number of run test cases in RichTest, the test execution time will also be reduced in this tool. But in order to accurately calculate the RT time for each project, it is necessary to calculate the overhead time due to the use of this tool and consider it in the calculation of the RT time.

Therefore, we made changes in the RichTest so that all the times related to doing the general tasks, segmentation, and creating connections between code and test blocks are calculated and stored in the project database. For four projects, we calculated and recorded the overhead time in RichTest, then we added these

time to the RT time and compared the result with the RT time in the TDD and S TDD methods. The final results are presented in Table 10.

Table 10: Comparison of the Number of Run Test Cases and RT Time in TDD,STDD, and RichTest (*considering RichTest overhead time*) for Four GitHub Projects

Project	Number of Run Test Cases			RT Time			Number of test cases	Number of versions
	TDD	STDD	RichTest	TDD	STDD	RichTest		
Junit-dataprovider	12312	4437	750	42379	12821	2756	(1 - 240)	69
Commons-Math	10106	2061	501	2517	132	222	(9-226)	70
ScribeJava	4485	257	157	12479	537	484	(51-66)	69
Jedis	5432	1106	382	13706	1031	1073	(10-136)	53

As can be seen in Table 10 and Fig. 14, the RichTest RT time improved more than 10 times compared to the TDD method. The logarithmic vertical axis in Fig. 14 represents that the time has improved tenfold. We also compared RichTest RT time (including overhead time) with STDD. It was found that they have slight differences with each other.

If there are a few test cases written for each class, the number of selected test cases in both methods is almost the same and as a result, STDD is slightly faster than RichTest. But if there are a lot of test cases, our tool selects only the related test cases and will perform better despite the overhead time. Also, results show that RichTest is suitable for large projects. Because in the early versions, the number of selected test cases and RT time does not differ much.

At last, by using Python's autorank[20] function [43], we compare the number of test cases and RT time between RichTest with TDD and STDD for four projects (other projects were not completely applicable). Final results are shown in Table 11-Table 14.

Table 11: Summary of comparison of number of run test cases in RichTest and TDD (produced by AutoRank)

Project	sample	meanrank		median		mad[21]		ci_lower		ci_upper		effect_size		m
		RichTest	TDD	RichTest	TDD	RichTest	TDD	RichTest	TDD	RichTest	TDD	RichTest	TDD	
Junit-dataProvider	68	1/99	1/01	4	201/5	3	72/5	3	105	8	255/00	0	-2/60	ne
Commons-Math	69	1/99	1/01	6	128	3	89	4	83	9	248	0	-1/31	ne
Jedis	52	1/99	1/01	3	111	1	15/5	2	100	8	127	0	-6/63	ne
Commons-Lang	23	1/98	1/02	2	138	1	3	1	117	20	140/00	0	-41/02	ne

Table 12: Summary of comparison of number of run test cases in RichTest and STDD (produced by AutoRank)

Project	sample	meanrank		median		mad		ci_lower		ci_upper		effect_size	
		RichTest	STDD	RichTest	STDD	RichTest	STDD	RichTest	STDD	RichTest	STDD	RichTest	STDD
Junit-dataProvider	68	1/96	1/04	4/00	55/50	3/00	26/50	3/00	39/00	8/00	89/00	0	-1/84
Commons-Math	69	1/96	1/04	6/00	24/00	3/00	12/00	4/00	15/00	9/00	37/00	0	-1/39
Jedis	52	1/98	1/02	3/00	15/50	1/00	9/50	2/00	8/00	8/00	26/00	0	-1/25
Commons-Lang	23	1/93	1/07	2/00	14/00	1/00	12/00	1/00	2/00	20/00	38/00	0	-0/95

Table 13: Summary of comparison of RT Time in RichTest and TDD (produced by AutoRank)

Project	sample	meanrank		median		mad		ci_lower		ci_upper		effect_size	
		RichTest	TDD	RichTest	TDD	RichTest	TDD	RichTest	TDD	RichTest	TDD	RichTest	TDD
Junit-dataProvider	68	1/98	1/02	16/17	713/51	9/08	249/26	10/44	352/17	29/90	877/13	0	-2/67
Commons-Math	69	1/99	1/01	2/37	27/50	1/25	21/01	1/60	19/62	3/93	63/35	0	-1/14
Jedis	52	1/98	1/02	10/51	287/92	4/35	45/29	7/26	245/18	20/74	329/84	0	-5/82
Commons-Lang	23	1/91	1/09	0/94	24/05	0/48	0/56	0/53	20/78	11/90	24/47	0	-29/8

Table 14: Summary of comparison of RT Time in RichTest and STDD (produced by AutoRank)

Project	sample	meanrank		median		mad		ci_lower		ci_upper		effect_size	
		RichTest	STDD	RichTest	STDD	RichTest	STDD	RichTest	STDD	RichTest	STDD	RichTest	STDD
Junit-dataProvider	68	1/11	1/89	16/17	14/49	9/08	8/06	10/44	10/14	29/90	25/67	-0/13	0
Commons-Math	69	1/12	1/88	2/37	1/14	1/25	0/55	1/60	0/82	3/93	1/85	1/85	0
Jedis	52	1/10	1/90	10/51	9/88	4/35	3/96	7/26	7/26	20/74	20/45	-0/10	0
Commons-Lang	23	1/02	1/98	0/94	0/37	0/48	0/18	0/53	0/19	11/90	3/28	-1/06	0

The statistical analysis was conducted for 2 populations with 68 paired samples.

The family-wise significance level of the tests is $\alpha=0.050$.

We rejected the null hypothesis that the population is normal for the populations STDD time ($p=0.000$) and RichTest time ($p=0.000$). Therefore, we assume that not all populations are normal.

No check for homogeneity was required because we only have two populations.

Because we have only two populations and both of them are not normal, we use Wilcoxon's signed rank test to determine the differences in the central tendency and report the median (MD) and the median absolute deviation (MAD) for each population.

We reject the null hypothesis ($p=0.000$) of Wilcoxon's signed rank test that population STDD time ($MD=15.095+6.889$, $MAD=7.954$) is not greater than population RichTest time ($MD=15.329+8.124$, $MAD=8.590$). Therefore, we assume that the median of STDD time is significantly larger than the median value of RichTest time with a negligible effect size ($\gamma=-0.019$).

Considering that the initial versions of the projects are also taken into account, RichTest RT time is worse than STDD method, but the number of run test cases in all projects shows the superiority of RichTest. Also, RichTest RT time is better than TDD in every four projects. Magnitude fields shows that RichTest is negligible while TDD and STDD are large.

6.4 Research Questions

This study focuses on the following five main questions:

RQ1: How many (complexity) test cases would be executed in the traditional and improved TDD process? The question is, if n test cases are written during the TDD process, what is the complexity of the number of test cases that will be executed?

6.4.1 Calculation of Minimum Number of Test Case Execution

In test-driven development, all previous tests should be re-executed in each iteration to ensure that they will perform correctly under new conditions. Among the principal disadvantages of TDD is the necessity of having a large number of test cases that must be repeatedly executed.

For clarity, consider the following example. Suppose that n is the number of test cases, which have been written and passed one by one, during the program development. This means that every time a new test is added, all previous tests, including the first test, are run again. Therefore, the first test will be performed at least n times. The second, third, and n th tests are executed at least $(n-1)$ times, at least $(n-2)$ times, and at least once, respectively.

So we add these items to get the minimum number of times that the test cases will have to be executed. As can be seen, the sum is equivalent to the sum of an arithmetic sequence that we have calculated by the formula (9).

$$\text{SUM } (1:n) = n + (n-1) + (n-2) + \dots + 3 + 2 + 1 = \frac{1}{2} n (n+1) \quad (9)$$

In formula (9) we supposed that the TDD development process starts with only one test, but some of the GitHub projects used in this research have more than one test case in the first version. So, we suppose that the initial number of test cases is t_1 (instead of one test), and the number of final tests is t_2 ; hence, there are t_1 , t_1+1 , and t_2 tests in the first, second, and last turns, respectively. The sum of the number of times the test cases are executed can be calculated by the formula (10).

$$\text{SUM}(t_1:t_2) = \frac{1}{2} (t_2 - t_1 + 1) * (t_1 + t_2) \quad (10)$$

As presented by formulae (9) and (10), the *minimum* number of times that the test cases are executed is calculated by the sum of an arithmetic sequence formula. So, the total number of test case executions is of $O(n^2)$ complexity, where n is the number of test cases. Actually, we have a quadratic complexity in traditional TDD, but in practice, we reach a linear complexity of executing test cases using RichTest, improved TDD (Fig. 12).

Considering that the number of test cases in the TDD is many times more than those in other methods, the relationship between the number of times that the tests will have to be executed and the second power of the number of test cases is one of the principal problems of TDD.

RQ2: How can we reduce the number of times that the test cases are executed without compromising the software reliability of TDD?

6.4.2 Safe Test Case Selection

The main problem in test case reduction methods is the lack of confidence that the reduced test suite can detect errors. If we can ensure that the selected test cases can detect all errors, then the method is safe as well as software quality and reliability are maintained.

For this purpose, we intend to delete only the insignificant test cases. Thus some of the test cases that are less important could be ignored execution in any interval. In this paper, we focus on the differences between the two versions of the program instead of focusing solely on its latest version. As presented in section 2.2.3, the behaviors of unchanged components in the new and old versions of a program do not differ at runtime so, it is guaranteed that no retest of all cases is necessary, and testing the affected component only is sufficient. RichTest skips all the test cases related to the unaffected parts of the program in RT. All test cases related to the modified parts are considered, so we have 100% modified code coverage.

The main problem in test case reduction methods is the lack of confidence that the reduced test suite can detect errors. If we can ensure that the selected test cases can detect all errors, then the method is safe as well as software quality and reliability are maintained.

Rothermel believed that under controlled RT, the modification-traversing tests are a superset of the fault-revealing tests [44]. Thus an algorithm that selects every modification-traversing test is also safe.

It should be mentioned that test $t \in T$ is modification-traversing for program P and modified program P' if and only if $ET(P(t))$ and $ET(P'(t))$ are nonequivalent. Execution trace $ET(P(t))$ for test t on program P , consisting of the sequence of statements in P that are executed when P is executed with t .

It should be mentioned that $ET(P(t))$ is the execution trace for test t on program P , consisting of the sequence of statements in P that are executed when P is executed with test t . Also, Test $t \in T$ is modification-traversing for program P and its modified program P' if and only if the execution traces of them are nonequivalent ($ET(P(t)) \neq ET(P'(t))$) [44].

What happens in our algorithm? Is it safe or not?

We "link" all the modified code blocks in each step to associated test cases. That is, when program P becomes P' , $ET(P(t))$ is different from $ET(P'(t))$. So we select all modification-traversing test cases.

When a test failure causes code modification, all modified code blocks are then connected to the test. Then all the tests related to these modified blocks are selected. Due to the code change in the block, for all of these selected tests, the sequence of executed instructions will be different at the time of running the test, i.e. $ET(P(t)) \neq ET(P'(t))$: These tests are all modification-traversing, and because they are a superset of the fault-revealing test suite, the algorithm is safe.

RQ3: How can the TDD method aid in selecting test cases?

6.4.3 TDD Based Test Case Selection

For the TDD method, the test case is written first, and thereafter the code is written to pass the test; a close relationship between the test case and source code is established. The question is whether the test cases can be selected based on the TDD *nature*.

We used the nature of TDD to model the relationship between test blocks and code blocks as shown in Fig. 15. In the TDD method, each requirement leads to writing a set of test cases. Each test case also leads to creation or modification of source code. So there are some relations between the test cases and modified parts of the code. That's why we use the code segmentation algorithm and save the relationship between test and code blocks

It should be mentioned that our selection algorithm is based on TDD nature and assumes that developers follow the TDD cycle. It may fail if the developer does not follow the TDD cycle, so another question arises.

RQ4: What is the impact of human behavior on this approach?

6.4.4 The Impact of Human Behavior

Our proposed approach assumes that developers always follow the TDD cycle. However, in reality, the order of this cycle is not always observed [44]. What is the impact of such a human behavior?

We assumed that the developer would not write any code except for passing the test or refactoring the code. Therefore, we connect all modified code blocks to the last test case. If the developer writes the code before writing the test case, RichTest assumes the changes are made to refactor the code. So RichTest connects these changes to the last test case.

Although ignoring the refactoring phase is not a problem, late refactoring may cause an unrealistic relationship between the previous code and the new test.

It is important to consider three questions. The first is whether the modified parts of the code are covered 100% or not. Fortunately, the answer is yes, because the modified parts will be connected to the last test case, and the coverage of the modified code is achieved.

The second question is whether the test cases will be selected correctly in the next steps. Unfortunately, the answer is "no". The test case that is mistakenly assigned to the code block may be selected and added to the test suite. The suggested solution is that the developer disconnects the wrong relations manually. This is possible through the BlockInfoView to uncheck and remove the test case relation.

The third question is whether the test suite is complete. Unfortunately, the answer is "no". Since the developer has not written any test case before modification, the test suite is not complete. The only suggested solution is that the developer connects the lost relations manually. This is possible through the BlockInfoView.

RQ5: To what extent it is possible to select test cases (semi-)automatically?

6.4.5 Automatic Test Case Selection

One of the main questions of the research is whether an effective model and tool can be considered to select test cases. Can a set of rules and steps that can be automated or semi-automated be defined to perform the task of the test case selection?

As explained in the previous question, RichTest is implemented based on the nature of TDD. In this way, if the new test fails, the programmer will apply enough changes to the code blocks, until passing the new test. Then, RichTest automatically links all the modified code blocks to the new test. Therefore, all test cases involved in creating or modifying any code block are linked to it.

During the development process, whenever each code block changes, RichTest selects all the test cases associated with that code block as a candidate test case. Therefore, any test case that was involved in creating or modifying the code will be selected automatically. Also, the programmer could link/unlink a test block to a code block manually.

[14] result = autorank(data, alpha=0.05, verbose=False)

[15] median absolute deviation

[16] <https://github.com/scribejava/scribejava>

[17] <https://github.com/searls/jasmine-maven-plugin>

[18] <https://github.com/avaje-metrics/metrics>

[19] <https://github.com/xetorthio/jedis>

[20] <https://github.com/apache/commons-math>

[21] <https://github.com/TNG/junit-dataprovider>

7 Summary And Conclusion

7.1 Summary

In this research, the problem of excessive numbers of test cases developed in the TDD and the repetitive execution of test cases are investigated. The results indicate that the complexity of test case execution correlates with the second power of the number of test cases.

The differences between the two program versions while ignoring test cases related to unmodified parts are identified, and some insights to reduce test cases and RT execution time in the TDD are suggested. A combinational difference identification algorithm based on textual and syntactical differencing is thereafter presented to accomplish these tasks. The proposed method to reduce test cases, particularly for the TDD method is presented. Program differencing is not a new approach to test selection, but the innovative aspect of our work is "how" to do it. We select test cases using the nature of TDD. For this purpose, we developed the RichTest tool. Whenever a copy of the program is saved, RichTest considers this version as a commit of the program and automatically monitors new test cases and program differences as new test blocks and modified/ new code blocks, then establishes the relationship between test cases and code blocks, automatically. TDD-based RT selection performs using these connections.

The RichTest plugin is employed to improve and simplify the implementation of TDD projects by reducing the number of run test cases and also reducing the RT execution time. The RT is executed by selecting only the test cases related to modified code blocks.

To evaluate the improved method, three small programs and six large open-source projects on GitHub are implemented in RichTest. The results show the RichTest plugin significantly reduces the number of test case executions by reducing the number of selected test cases at runtime (compared to both TDD and STDD). This reduction would be more significant in large projects with a larger number of test cases. Also, the number of times that the test cases runs in

RichTest completely overlaps with the number of test cases. Although we have a quadratic complexity in traditional TDD, but in practice, we reach a linear complexity of executing test cases using RichTest, improved TDD.

The results showed that in the first version of each project, the number of test cases in pure TDD, STDD and RichTest is equal to the total number of test cases, so in the first version the RichTest method has the longest execution time due to the overhead time; but gradually by reducing the number of selected test cases in the next versions, this overhead time will be compensated and the total execution time will be reduced. RichTest RT time (including overhead time) is one tenth of TDD RT time. It was found that RichTest and STDD RT time have slight differences with each other. If there are a few test cases written for each class, the number of selected test cases in both methods is almost the same and as a result, STDD is slightly faster than RichTest. But if there are a lot of test cases, RichTest selects only the related test cases and will perform better despite the overhead time.

7.2 Restrictions

RichTest is not a commercial tool and is only the result of student research, so it is not free of problems and limitations. Its limitations are presented below.

1. Our block segmentation algorithm is based on Java programming language grammar, so RichTest limits projects to Java language. Also only some of control flow instructions such as switch, while, if, for, foreach, and try are considered.
2. Our plugin is developed in Eclipse IDE Photon June 2018, so RichTest limits to this development environment and Junit4 Tests.
3. Our method supports only TDD projects that follow the TDD cycle, otherwise, the developer must manually (dis)connect the code blocks (from) to the related tests. Human behavior is explained in Section 5.4.4 in more detail.
4. Our plugin doesn't execute test cases properly on the maven projects and gradle projects. Sometimes test case execution encountered a problem and we had to write another program to run the test cases.
5. This plugin is not recommended for projects that have many interfaces, because the number of selected test cases will not decrease significantly. Interface modification propagates to all of its implementations, so all tests related to all those codes should be selected for re-execution.
6. The RichTest tool uses commenting to track each block. For example, the beginning and the end of the first code block are defined by inserting two comments: //Start Of Code-Block: ca001, and //End Of Code-Block: ca001, respectively. RichTest needs these comments to trace code blocks, so in the refactoring phase, it is necessary to keep the comments in place so that the connections between the previous code blocks and related test cases are retained. Removing these comments will disrupt the test case selection process.

Future Work

We are going to upgrade the plugin to resolve some of the restrictions, provide execution time reports, and keep the test result history. We will use these reports to prioritize tests. For example, the test face more failures will have a higher priority. We use this result to combine our test selection algorithm with regression test prioritization. Also, we should investigate another real project as well as start a real project implementation in a laboratory.

References

1. Beck, K.: Test Driven Development: By Example. Addison-Wesley, Boston (2002)
2. Fowler, M., Beck, K., Brant, J., Opdyke, W., Roberts, D.: In: Gamma, E. (ed.) Improving the Design of Existing Code, 1st edn. Karnataka: Pearson Education India, Refactoring (1999)
3. Khanam, Z., Mohammed, N.A.: Evaluating the effectiveness of test driven development: advantages and pitfalls. Int. J. Appl. Eng. Res. **12**(18), 7705–7716 (2017)
4. Erdogmus, H., Maurizio, M., Marco, T.: On the effectiveness of the test-first approach to programming. IEEE Trans. Software Eng. **31**(3), 226–237 (2005)
5. Ammann, P., Offutt, J.: Introduction to Software Testing. Cambridge University Press, Cambridge (2008)
6. Beningo, J.: Testing, Verification, and Test-Driven Development. In: Embedded Software Design: A Practical Approach to Architecture, Processes, and Coding Techniques, pp. 197–218. Apress, Berkeley, CA (2022)
7. Astels, D.: Test Driven development: A Practical Guide. Prentice-Hall/Pearson Education, New Jersey (2003)
8. Riebisch, M., Farooq, Q., Lehnert, S.: "Model-Based Regression Testing: Process, Challenges and Approaches," in *Emerging Technologies for the Evolution and Maintenance of Software Models*, Ilmenau, Germany, IGI Global, pp. 254–297. (2012)
9. Rosero, R.H., Omar, G.S., Glen, R.: 15 Years of Software Regression Testing Techniques – A Survey. Int. J. Software Eng. Knowl. Eng. **26**(05), 675–689 (2016)
10. Yoo, S., Mark, H.: "Regression testing minimization, selection and prioritization: a survey," *Software testing, verification and reliability*, vol. 22, no. 2, pp. 67–120, (2012)
11. Zhang, L., Hao, D., Zhang, L., Rothermel, G.: "Bridging the gap between the total and additional test-case prioritization strategies," in *Proceedings of the International Conference on Software Engineering*, San Francisco, 2013. (2013)
12. Rothermel, G., Mary, J.H.: Empirical studies of a safe regression test selection technique. IEEE Trans. Software Eng. **24**(6), 401–419 (1998)
13. Apiwattanapong, T., Orso, A., Harrold, M.J.: JDiff: A differencing technique and tool for object-oriented programs. Automated Softw. Eng. **14**(1), 3–36 (2007)

14. Binkley, D.: "Using semantic differencing to reduce the cost of regression testing," in *IEEE Conference on Software Maintenance*, Orlando, (1992)
15. Asaduzzaman, M., Roy, C., Schneider, K., Di Penta, M.: "LHDiff: A language-independent hybrid approach for tracking source code lines," in *IEEE International Conference on Software Maintenance*, Eindhoven, (2013)
16. Myers, E.W.: "AnO (ND) Difference Algorithm and Its Variations," *Algorithmica*, vol. 1, no. 1–4, pp. 251–266, (1986)
17. Vokolos, F.I., Frankl, P.: "Empirical evaluation of the textual differencing regression testing technique," in *IEEE International Conference on Software Maintenance (Cat. No. 98CB36272)*, Bethesda, (1998)
18. Canfora, G., Cerulo, L., Penta, M.D.: "LDiff: an Enhanced Line Differencing Tool," in *31st International Conference on Software Engineering. IEEE Computer Society*, Washington, (2009)
19. Yang, W.: Identifying syntactic differences between two programs. *Software: Pract. Experience*. **21**(7), 739–755 (1991)
20. Maletic, J.I., Collard, M.L.: "Supporting Source Code Difference Analysis," in *20th IEEE International Conference on Software Maintenance*, Chicago, (2004)
21. Archambault, D.: "Structural differences between two graphs through hierarchies," in *Proceedings of Graphics Interface*, Kelowna, (2009)
22. Goto, A., Yoshida, N., Ioka, M., Choi, E., Inoue, K.: "How to extract differences from similar programs? A cohesion metric approach," in *7th International Workshop on Software Clones (IEEE Press)*, San Francisco, (2013)
23. Linares-Vásquez, M., Cortés-Coy, L., Aponte, J., Poshypanyk, D.: "Changscribe: A tool for automatically generating commit messages," in *37th IEEE International Conference on Software Engineering*, Florence, (2015)
24. Shen, J., Sun, X., Li, B., Yang, H., Hu, J.: "On automatic summarization of what and why information in source code changes," in *40th Annual Computer Software and Applications Conference (COMPSAC)*, Atlanta, (2016)
25. Kim, M., David, N.: "Discovering and representing systematic code changes," in *IEEE 31st International Conference on Software Engineering*, Washington, (2009)
26. Falleri, J., Floréal, M., Xavier, B., Matias, M., Martin, M.: "Fine-grained and Accurate Source Code Differencing," in *29th ACM/IEEE international conference on Automated software engineering*, Västerås, (2014)
27. Wang, X., Pollock, L., Vijay-Shanker, K.: "Automatic segmentation of method code into meaningful blocks to improve readability," in *18th Working Conference on Reverse Engineering IEEE*, Limerick, (2011)
28. Horwitz, S.: Identifying The Semantic and Textual Differences Between Two Versions of a Program. *ACM sigplan*. **25**(6), 234–245 (1990)
29. Neamtiu, I., Foster, J.S., Hicks, M.: "Understanding Source Code Evolution Using Abstract Syntax Tree Matching," Missouri, (2005)
30. Görg, M., Zhao, J.: "Identifying semantic differences in AspectJ programs," in *18th international symposium on Software testing and analysis (ACM)*, Chicago, (2009)
31. Hsu, R.: "Method for detecting differences between graphical programs".U.S. Patent5,974,254, 26 Oct 1999.
32. Debin, G., Reiter, M.K., Song, D.: "Binhunt: Automatically finding semantic differences in binary programs," in *International Conference on Information and Communications Security. Springer*, Berlin, Heidelberg, (2008)
33. Wang, T., Wang, K., X. SU and, MA, P.: Detection of semantically similar code. *Front. Comput. Sci*. **8**(6), 996–1011 (2014)
34. Liu, C., Chen, C., Han, J., Yu, P.S.: "GPLAG: Detection of Software Plagiarism by Program Dependence Graph Analysis," in *12th ACM SIGKDD international conference on Knowledge discovery and data mining*, New York, (2006)
35. Nguyen, H.A., Nguyen, T.T., Nguyen, H.V., Nguyen, T.N.: "idiff: Interaction-based program differencing tool," in *26th IEEE/ACM International Conference on Automated Software Engineering*, Washington, (2011)
36. Madeyski, L., Marcin, K.: "Continuous Test-Driven Development - A Novel Agile Software Development Practice and Supporting Tool,"Evaluation of Novel Approaches to Software Engineering (ENASE), pp.260–267, (2013)
37. Madeyski, L., Kawalerowicz, M.: Continuous Test-Driven Development: A Preliminary Empirical Evaluation Using Agile Experimentation in Industrial Settings. Towards a Synergistic Combination of Research and Practice in Software Engineering. Springer. **733**, 105–118 (2018)
38. Cibulski, H., Amiram, Y.: "Regression Test Selection Techniques for Test-Driven Development," in *IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*, Washington, (2011)
39. Santosh Singh, R., Kumar, S.: An approach for the prediction of number of software faults based on the dynamic selection of learning techniques. *IEEE Trans. Reliab*. **68**(1), 216–236 (2018)
40. Biswas, S., Mall, R., Satpathy, M., Sukumaran, S.: "Regression Test Selection Techniques: A Survey," *Informatica*, vol. 35, no. 3, pp. 289–321, (2011)
41. Wolfgang, O.: "TDD Kata," 9 12 2018. [Online]. Available: <https://www.programmingwithwolfgang.com/tdd-kata/>. [Accessed 8 5 2021]
42. Borle, N.C., Fegghi, M., Stroulia, E., Greiner, R., Hindle, A.: Analyzing the effects of test driven development in GitHub. *Empir. Softw. Eng*. **23**(4), 1931–1958 (2018)
43. Herbold, S.: Autorank: A python package for automated ranking of classifiers. *J. Open Source Softw*. **5**(48), 2173 (2020)
44. Rothermel, G., Harrold, M.J.: A safe, efficient regression test selection technique. *ACM Trans. Softw. Eng. Methodol. (TOSEM)*. **6**(2), 173–210 (1997)
45. Beller, M., Georgios, G., Annibale, P.: Developer testing in the ide: Patterns, beliefs, and behavior. *IEEE Trans. Software Eng*. **45**(3), 261–284 (2017)

Figures

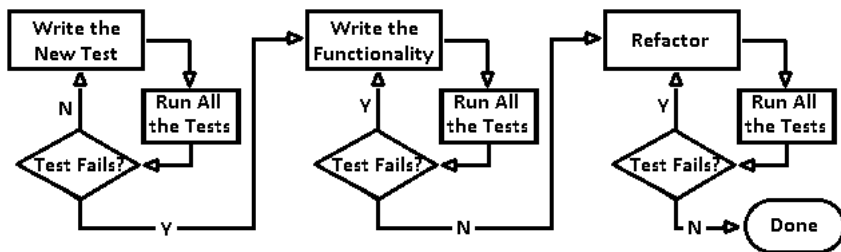


Figure 1

TDD Activities [36]

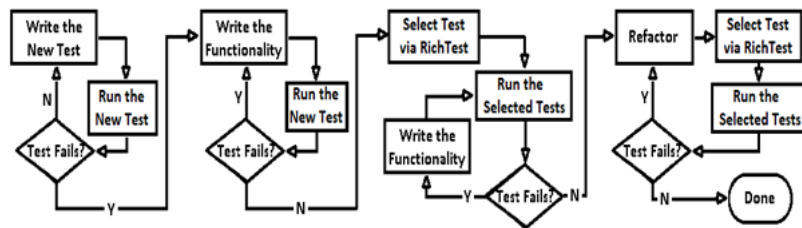


Figure 2

Our Improved TDD Activities to Reduce Test cases

The screenshot shows the Eclipse IDE with the following components:

- SortClass.java:**

```

1 package SortPackage;
2
3 public class SortClass {
4     //Start Of Code-Block : ca002
5     public static String stupidsort(int[] x, int len)
6     {
7         String status="Call Sort";
8         int t;
9         int i=0;
10        //Start Of Code-Block : ca003
11        if (len > 1) {
12            //Start Of Code-Block : ca004
13
14            //Start Of Code-Block : ca005
15            while(i < len-1){
16                //Start Of Code-Block : ca007

```
- Regression Tests View:**

ID	Regression Test Name	Total Time
17	Quick-Regression-Test-2021-09-04-...	55
16	TODAY 1111111	107
15	today test 1	0

Name	Time	Result
ta007	20	Fail

Overhead: 37ms
Total Time: 57ms

Buttons: Run All Tests, Run With Priority

Figure 4

Use the RichTest plugin in Eclipse for the sort program

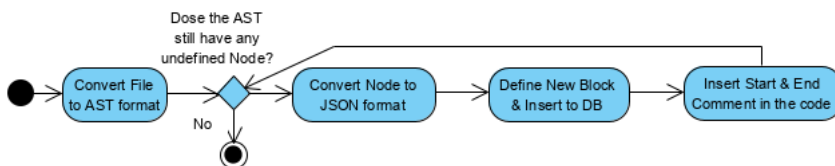


Figure 5

Automatic Code Block Segmentation

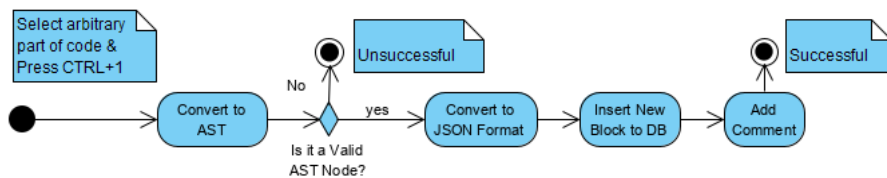


Figure 6

Manual Code Block Segmentation

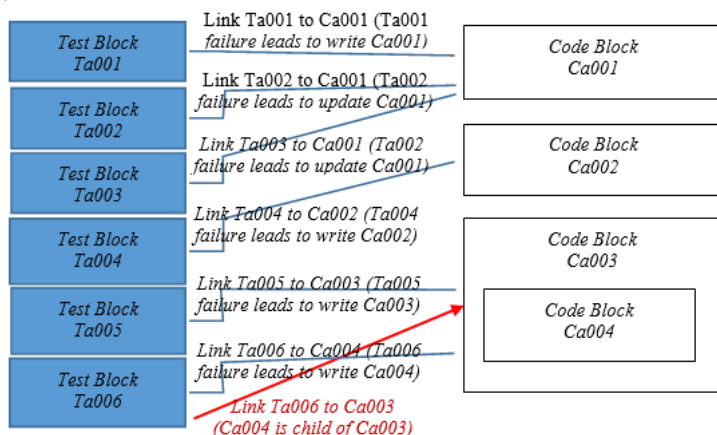


Figure 7

Example of n:n Relationship between Test Blocks and Code Blocks

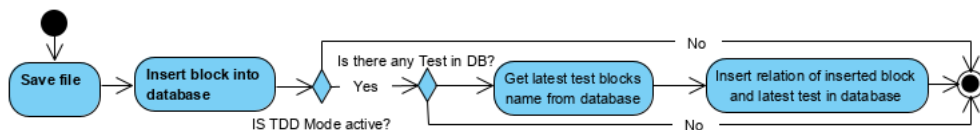


Figure 8

Relationship between Modified Code Blocks and New Test Block

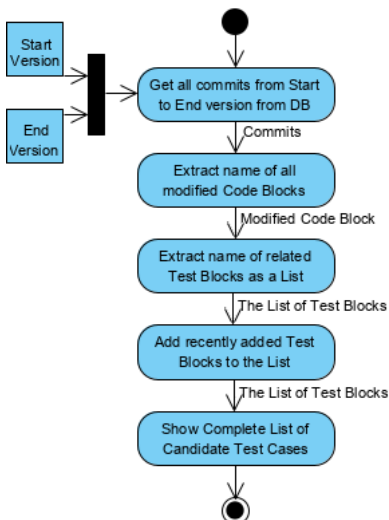


Figure 9

RichTest Test Selection Process Sequence Diagram

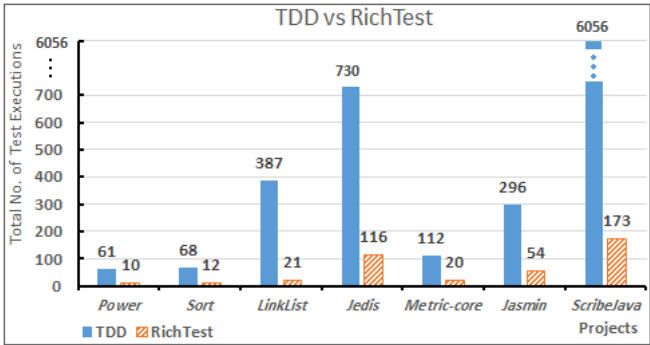


Figure 10

Total number of Test Case Executions in TDD vs RichTest

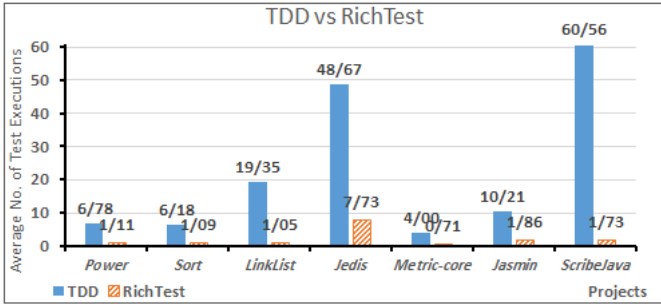


Figure 11

Average of Test Case Execution for each Iteration in TDD vs RichTest

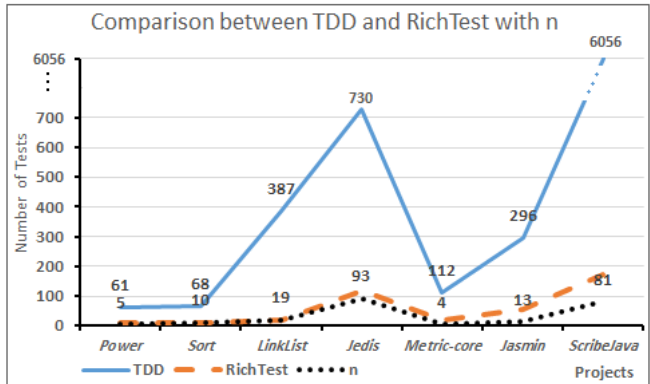


Figure 12

Comparison between TDD and RichTest with n

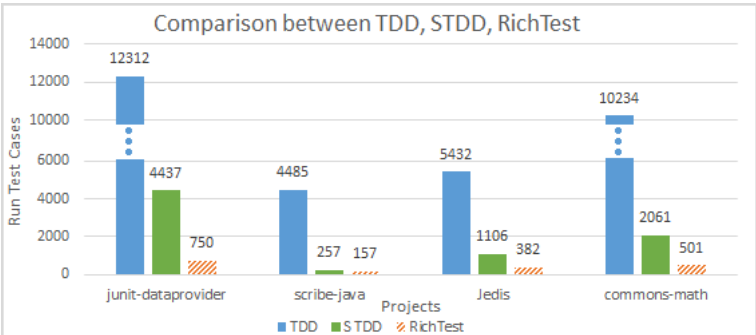


Figure 13

Comparison between Run Test cases in TDD, STDD, and RichTest for four GitHub Projects represented in Table 9

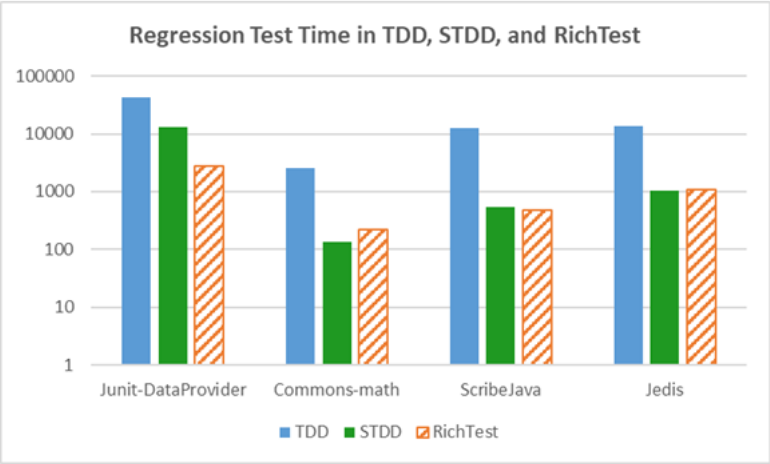


Figure 14
Comparison of RT Time in TDD, Simple TDD, and RichTest for Four GitHub Projects represented in Table 10

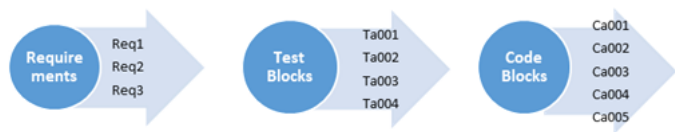


Figure 15
Relationship between Requirement, Test, and Code

Supplementary Files

This is a list of supplementary files associated with this preprint. Click to download.

- [APPENDIXA.docx](#)